

TOWARDS REVERSIBLE MODEL MERGING FOR LOW-RANK WEIGHTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Model merging seeks to synthesize multiple fine-tuned models into a single parameterization whose behavior remains competitive across all source tasks. Although prior work has established that merged weights can approximate the task-wise performance of the corresponding specialized models, this literature has paid comparatively little attention to regimes in which the constituent models have themselves been represented in low-rank form, either through low-rank adaptation (LoRA) or through post-training singular value decomposition (SVD). We first show that, when conventional merging procedures are applied directly to such low-rank weights, the resulting merged model undergoes pronounced performance deterioration. Motivated by this empirical pathology, we propose a substantially different formulation: rather than collapsing all adapters into one weight tensor, we construct a compact basis (e.g., an equivalent of holding two or more models) from which the original task-specific models can be recovered by linear combination. This perspective recasts merging as the construction of a reconstruction-capable model space, rather than as the production of a single terminal model. Crucially, it permits controlled “reversion” to each individual model when necessary, reflecting the fact that no merged model can be expected to uniformly dominate a model specialized for its own task. On this basis, we introduce Reversible Model Merging (RMM), an efficient, data-free, and flexible method that yields a closed-form solution for selecting the optimal weight basis together with the task-specific coefficients used for linear recombination. Extensive experiments across heterogeneous datasets and model scales demonstrate that RMM consistently surpasses existing merging approaches, preserving the performance of low-rank compressed models by a substantial margin.

1 INTRODUCTION

Parameter-efficient fine-tuning methods, most notably Low-Rank Adapters (LoRA) (Hu et al., 2021), have become central to the adaptation of large language models across heterogeneous downstream tasks without the computational burden of full retraining. In parallel, model compression techniques, including post-training singular value decomposition (SVD) truncation, have been widely adopted as mechanisms for reducing storage and inference costs, thereby introducing an additional axis of efficiency (Ryu et al., 2023; Ping et al., 2024; Wang et al., 2025b;a). Against this backdrop, model merging has emerged as an attractive paradigm for consolidating the knowledge encoded in multiple task-specific models into a single unified parameterization. Recent merging methods, spanning simple parameter interpolation through optimization-based procedures, have demonstrated that merged models can remain competitive across source tasks even when the original training data are unavailable (Matena & Raffel, 2022; Yang et al., 2024; Yadav et al., 2023; Wei et al., 2025).

Notwithstanding this progress, a consequential limitation has remained insufficiently examined: even sophisticated merging strategies frequently fall short of the performance attained by the original, individually fine-tuned models. The discrepancy becomes especially acute when the models to be merged have already been compressed into low-rank form, whether through LoRA fine-tuning or post-training SVD truncation. Although merging enables a single model to serve multiple tasks, it necessarily projects distinct task-specific representations into one shared parameter space, a consoli-

idation that can induce cross-task interference and attenuate the specialized adaptations acquired by each constituent model (Gargiulo et al., 2025).

Although low-rank adapters and SVD-truncated models are individually compact, their accumulation produces a distinct scalability bottleneck. In realistic multi-task, federated, or continual-learning deployments, dozens or even hundreds of task-specific low-rank adapters may be produced, each of which entails separate storage, indexing, and management. Retaining this entire collection progressively erodes the storage advantage that low-rank parameterization was intended to provide and complicates deployment. These considerations motivate merging strategies that operate directly on low-rank representations, not merely to reduce redundancy and consolidate storage, but also to facilitate more efficient sharing, deployment, and transfer of compressed models across tasks.

To characterize the limitations of existing approaches, we first examine how standard merging techniques behave when applied to low-rank compressed models. Our analysis focuses on deltas, namely the weight differences between each fine-tuned model and the shared pre-trained initialization. Conventional merging procedures attempt to combine these deltas into a single parameter set under an implicit assumption of linear compatibility. Empirically, however, we find that direct merging of low-rank deltas induces catastrophic degradation. This failure arises from two interacting sources: the restricted expressivity inherent to low-rank representations and the misalignment of independently optimized task-specific subspaces, whose combination produces severe interference.

This observation motivates a more fundamental reconsideration of the objective itself: *What if the purpose of merging need not be the production of a single final model? What if the storage budget could permit more than one retained component? What if the post-merge representation could preserve the capacity to reconstruct the original models on demand?* Rather than compressing multiple low-rank adapters into one weight set, we propose to maintain a compact collection of weights that functions as a shared basis from which each original model can be recovered through linear combination. This reformulation exposes an underexplored design space: instead of forcing all information into a single model, we preserve complementary representational directions across a small set of basis models, thereby enabling post-merging “reversion” to the original adapters. The resulting view challenges the prevailing assumption that merging must terminate in one model and instead defines merging as the construction of a reconstruction-capable model space.

Building on this reconstruction-oriented perspective, we develop a theoretical framework that characterizes the basis minimizing reconstruction error. This framework yields Reversible Model Merging (RMM), which interprets merging as a basis-selection problem rather than as direct parameter aggregation. Specifically, the objective is to identify a compact set of task-vector directions from which all original models can be accurately reconstructed. Within this formulation, RMM admits a closed-form solution that jointly determines the optimal basis and the task-specific linear-combination coefficients required to recover individual adaptations. We empirically validate RMM across a broad range of datasets and model configurations, showing that it consistently surpasses conventional merging strategies by a significant margin while preserving the efficiency benefits of low-rank parameterization. Moreover, through a tunable hyperparameter controlling the number of retained final components, RMM offers a flexible trade-off between storage and performance, underscoring its practicality for large-scale multi-task adaptation.

2 RELATED WORK

Model merging has become an influential strategy for consolidating knowledge distributed across multiple fine-tuned models. Existing methods can be broadly organized into two families: data-free manipulations performed directly in parameter space and optimization-based approaches that incorporate data or auxiliary objectives. We review representative techniques from both lines of work below:

Task Arithmetic: Task arithmetic constitutes one of the foundational formulations of model merging (Ilharco et al., 2023). In this approach, a delta is defined as the parameter difference between a fine-tuned model and its pre-trained initialization, and the central premise is that such deltas behave as modular units of behavioral change. They can therefore be added to a base model to introduce new capabilities, subtracted to attenuate undesired behaviors, or scaled and combined to interpolate

among multiple skills. This intuitive linear calculus established an important basis for composing model behavior directly in weight space.

DARE: Although simple delta addition can be effective, it often degrades when the constituent models contain conflicting or overlapping parameter updates. To mitigate this issue, Yu et al. (2024) introduced DARE (Drop and Rescale), a technique based on aggressive randomized pruning. The method discards the overwhelming majority of parameter changes—up to 90-99%—under the premise that many updates are nonessential and may interfere with knowledge contributed by other models. The surviving updates are then amplified to preserve the global magnitude of the modification, thereby retaining core task-specific changes while reducing interference.

TIES: In contrast to DARE’s stochastic pruning, TIES-merging provides a deterministic framework for mitigating parameter conflicts (Yadav et al., 2023). It proceeds through three phases—Trim, Elect Sign, and Merge—each of which is designed to resolve cross-model interference systematically. First, the method retains only the largest-magnitude parameters, thereby preserving the updates that fine-tuning appears to have made most salient. It then evaluates, at each parameter position, the signs of the updates across models and enforces agreement by discarding contributions that contradict the majority sign. Finally, the sign-consistent parameters are averaged to obtain the merged representation. This consensus-driven procedure reduces destructive interference and produces a more stable merged model. Pursuing a similar objective, EMR-Merging (Elect, Mask and Rescale) elects a unified model and attaches task-specific binary masks and rescalers derived from each individual model, thereby shifting the unified model toward each task (Huang et al., 2024).

Other Merging Methods: Beyond purely parameter-space manipulations, several merging approaches depend on optimization procedures or access to a small auxiliary dataset. Model soups (Wortsman et al., 2022), for instance, refine task-arithmetic weights by minimizing validation loss, thereby steering interpolation toward improved generalization. Fisher-weighted averaging (Matena & Raffel, 2022) similarly injects second-order information into the merge by assigning greater importance to parameters estimated to be more consequential for task performance. Other methods formulate merging explicitly as an optimization problem: RegMean (Jin et al., 2023) and AdaMerging (Yang et al., 2024), for example, adaptively search for task-dependent merging coefficients by solving small-scale objectives based, respectively, on the discrepancy between merged-model and individual-model predictions and on the entropy of predictions over unlabeled data. More recently, model merging has been cast as minimization of the loss difference between the merged and individual fine-tuned models, with a first-order Taylor expansion yielding a tractable surrogate objective (Wei et al., 2025). Although such approaches can improve task-specific performance, their reliance on labeled or unlabeled data, additional optimization, or both constrains their usefulness when merging must be conducted without retraining and without data access.

3 PRELIMINARIES

We study the consolidation of a collection of fine-tuned models into a unified model through model merging. Let $\{\theta_1, \dots, \theta_n\}$ denote n models fine-tuned on n distinct tasks. A merging algorithm $\mathcal{M}$ maps this collection to a single weight configuration, $\theta_{\mathcal{M}} = \mathcal{M}(\{\theta_i\}_{i=1}^n)$. The corresponding objective is to identify a procedure $\mathcal{M}$ that maximizes average validation performance over the full task set:

$$\max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\theta_{\mathcal{M}}, \mathcal{D}_i), \quad (1)$$

where $\mathcal{D}_i$ is a held-out validation set for task i , and Perf denotes an appropriate task metric, such as accuracy or F1 score. We adopt the standard assumption that all fine-tuned models share a common pretrained initialization θ_{pre} . For any layer l , the update, or delta, of model i relative to this base model is $\Delta_i^l = \theta_i^l - \theta_{\text{pre}}^l$, where $\Delta_i^l \in \mathbb{R}^{m \times d}$ represents the layer-wise weight difference for dimensions $m \times d$.

3.1 LOW-RANK COMPRESSED MODELS

In practice, fine-tuned models are often compressed into low-rank form in order to reduce storage, training, and inference costs. We focus on two widely adopted strategies:

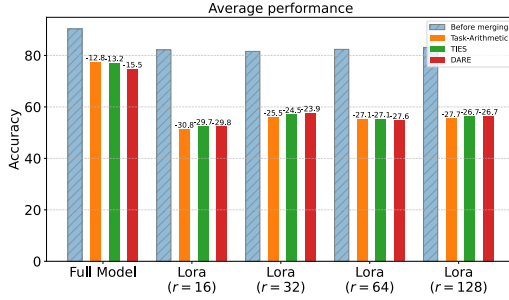


Figure 1: Average task performance for individual models prior to merging compared with merged models produced by different merging methods. Results are reported for fully fine-tuned models and for LoRA fine-tuned models across ranks $r \in \{16, 32, 64, 128\}$.

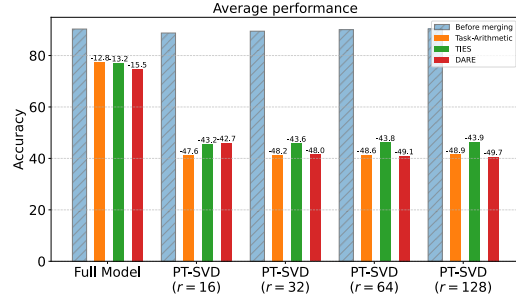


Figure 2: Average task performance for individual models prior to merging compared with merged models produced by different merging methods. Results are reported for fully fine-tuned models and for PT-SVD compressed models across ranks $r \in \{16, 32, 64, 128\}$.

LoRA fine-tuning: During training, each layer is augmented with trainable low-rank matrices $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times d}$, where $r \ll \min(m, d)$. The pretrained weights are kept fixed, while optimization is restricted to $\mathbf{A}$ and $\mathbf{B}$.

Post-training SVD truncation (PT-SVD): After standard fine-tuning, the update Δ_i^l is decomposed with SVD and truncated so that only the top- r singular components are retained, producing the low-rank approximation $\hat{\Delta}_i^l = \mathbf{A}_i^l \mathbf{B}_i^l$, with $\mathbf{A}_i^l \in \mathbb{R}^{m \times r}$ and $\mathbf{B}_i^l \in \mathbb{R}^{r \times d}$ for $r \ll \min(m, d)$.

Under either construction, the fine-tuned update for layer l in model i is represented as the low-rank product $\Delta_i^l = \mathbf{A}_i^l \mathbf{B}_i^l$, where $\mathbf{A}_i^l \in \mathbb{R}^{m \times r}$ and $\mathbf{B}_i^l \in \mathbb{R}^{r \times d}$, with r denoting the compression rank. The merging problem consequently becomes the problem of combining low-rank updates across tasks. The desired merged model integrates the compressed deltas $\hat{\Delta}_i = \{\hat{\Delta}_i^l\}_{l=1}^L$ over all tasks:

$$\hat{\theta}_{\mathcal{M}} = \theta_{\text{pre}} + \mathcal{M}(\{\hat{\Delta}_i\}_{i=1}^n), \quad \max_{\mathcal{M}} \frac{1}{n} \sum_{i=1}^n \text{Perf}(\hat{\theta}_{\mathcal{M}}, \mathcal{D}_i), \quad (2)$$

where L denotes the total number of layers. Throughout this work, we concentrate on this low-rank setting, in which every model has been compressed before merging. The next section examines how conventional merging strategies behave in this regime.

4 MOTIVATING OBSERVATION

We next examine the behavior of conventional model merging strategies when they are transferred to the low-rank compressed setting introduced above. For each layer l , prior work has considered two natural extensions of merging (Tang et al., 2025; Zhao et al., 2025; Zheng et al., 2025; Zhang et al., 2025):

- **Combined merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{\mathbf{A}_i^l \mathbf{B}_i^l\}_{i=1}^n),$$

where each low-rank update is first reconstructed into full-rank form before being merged.

- **Separate merging:**

$$\hat{\theta}_{\mathcal{M}}^l = \theta_{\text{pre}}^l + \mathcal{M}(\{\mathbf{A}_i^l\}_{i=1}^n) \mathcal{M}(\{\mathbf{B}_i^l\}_{i=1}^n),$$

where the low-rank factors are merged independently before recomposition.

Although both formulations are mathematically admissible, the combined strategy is practically inefficient. Storing the merged update in combined form requires materializing an $m \times d$ matrix, whereas the separate strategy stores only quantities comparable to the low-rank factors $\mathbf{A}_i^l \in \mathbb{R}^{m \times r}$ and $\mathbf{B}_i^l \in \mathbb{R}^{r \times d}$. Because $r(m + d) \ll md$ whenever $r \ll \min(m, d)$, the storage and memory

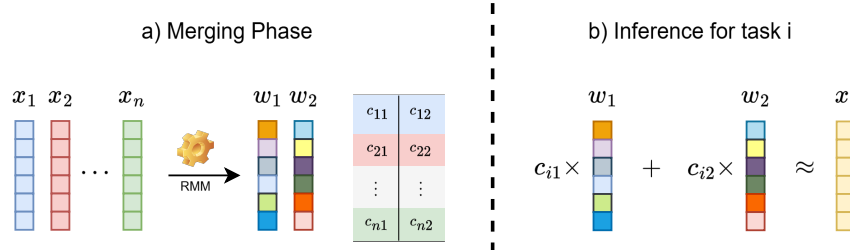


Figure 3: Overview of the proposed RMM framework. **Left:** During the merging phase, a compact set of basis directions (two in this illustration) and the associated task-specific coefficients are computed. **Right:** At inference time, target model i is reconstructed by linearly combining the basis vectors with the corresponding coefficients.

burden of the combined approach negates the intended benefit of compression. For a layer with $m = d = 1024$ and compression rank $r = 16$, for example, the combined approach stores a full update containing $md = 1,048,576$ parameters, whereas the separate approach requires only $r(m + d) = 16 \times (1024 + 1024) = 32,768$ parameters.

This corresponds to a **$32\times$ reduction in storage**, illustrating that the combined approach effectively forfeits the efficiency gains supplied by low-rank compression. We therefore focus on the separate strategy as the practically viable option for merging compressed models.

Using the separate approach described above, we applied several existing model-merging baselines to low-rank compressed models across varying compression ratios. RoBERTa-base (Zhuang et al., 2021) served as the pre-trained model, and the evaluation covered 5 GLUE tasks (Wang et al., 2018) (with additional details in Appendix A.3). As Figures 1 and 2 show, merging compressed models produces a *catastrophic drop* in average task performance relative to merging full-rank models. We attribute this degradation to two factors: (i) **Amplified task interference:** compression removes substantial representational capacity, making low-rank models more vulnerable to destructive interference during merging; (ii) **Loss of coupling between A and B :** conventional merging treats the low-rank matrices A_i^l and B_i^l as independent factors, disregarding the coupled structure of their product and thereby inducing structural misalignment in the merged updates. Together, these effects severely impair the performance of merged low-rank models and indicate a deeper incompatibility between traditional merging methods and compressed representations.

This observation yields a central insight: under conventional strategies, no single merged model can reliably recover the task-specific performance of the original low-rank adapters. It therefore becomes natural to ask: *Can the original compressed models instead be reconstructed, or closely approximated, by retaining a compact set of basis models whose size may correspond to two or more adapters depending on the storage budget?* Rather than forcing all updates into one representation, we investigate an alternative design space in which merging constructs a *reconstruction-capable model space*. We next formalize this perspective and introduce *Reversible Model Merging* (RMM).

5 REVERSIBLE MODEL MERGING

The empirical failures of conventional merging in the compressed setting point to a fundamental limitation: a single merged model cannot faithfully preserve the task-specific performance of all low-rank compressed models. Motivated by this limitation, RMM abandons the requirement that all models be collapsed into one weight set. Instead, it constructs a small collection of shared components, or basis models, from which all original task models can be approximately reconstructed.

Rather than forcing all n task-specific models into a single merged solution $\hat{\theta}_{\mathcal{M}}$, RMM retains a compact set of p merged components, with $2 \leq p \ll n$. These components define a low-dimensional subspace that spans the essential task variations and permits efficient recovery of each model on demand. The design therefore exposes a direct performance–memory trade-off: by adjusting p , practitioners can exchange storage cost for reconstruction accuracy.

For each layer l , we consider the r -dimensional vectors contained in the low-rank delta matrices (A_i^l and B_i^l), namely the rows of A_i^l and the columns of B_i^l , where r is the compression rank. Each row of A_i^l and each column of B_i^l is treated as an independent *task vector*, and a particular row or column index is called a *task vector position*. For any fixed position, we collect the corresponding task vectors across all n models and denote the vector from model i at that position by $\mathbf{x}_i \in \mathbb{R}^r$. The algorithm is then applied independently at each position (i.e., at a row index of A_i^l or a column index of B_i^l): the set $\{\mathbf{x}_i\}_{i=1}^n$ is compressed through projection into a shared basis so that reconstruction remains possible later. Since each layer contains $m + d$ task-vector positions, comprising m rows from A_i^l and d columns from B_i^l , the procedure is executed separately for each position.

To this end, we identify basis vectors $\{\mathbf{w}_1, \dots, \mathbf{w}_p\}$, with $\mathbf{w}_j \in \mathbb{R}^r$ for $1 \leq j \leq p$. Their span defines the subspace $\mathbf{W} = [\mathbf{w}_1 \dots \mathbf{w}_p] \in \mathbb{R}^{r \times p}$ into which each task vector $\mathbf{x}_i$ is projected, subject to the requirement that the resulting projection reconstructs $\mathbf{x}_i$ as accurately as possible. We introduce $\mathbf{C} \in \mathbb{R}^{n \times p}$ as the coefficient matrix, whose i -th row $[c_{i1}, \dots, c_{ip}]$ gives the scalar weights used to linearly combine basis vectors when reconstructing $\mathbf{x}_i$. The objective is therefore to choose $\mathbf{W}$ and $\mathbf{C}$ so that the aggregate reconstruction error over all n task vectors is minimized, which yields the following optimization problem:

$$\min_{\mathbf{W}, \mathbf{C}} \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^p c_{ij} \mathbf{w}_j \right\|_2^2. \quad (3)$$

This formulation stores rp parameters for the basis and np parameters for the coefficients, for a total of $p(r+n)$, which is substantially smaller than the rn parameters required to store every task vector explicitly. The value of p is consequently a tunable hyperparameter governing the performance-storage trade-off. Figure 3 illustrates the approach for $p = 2$. The optimization in equation 3 can be written in matrix form as follows:

$$\min_{\mathbf{W}, \mathbf{C}} \left\| \mathbf{X} - \mathbf{C} \mathbf{W}^\top \right\|_F^2, \quad (4)$$

where $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$ collects all task vectors. For any fixed $\mathbf{W}$, the optimal $\mathbf{C}$ is given by the least-squares solution $\mathbf{C}^* = \mathbf{X} \mathbf{W} (\mathbf{W}^\top \mathbf{W})^{-1}$. Under the orthonormality assumption $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$ with $p < r$, the optimization can therefore be rewritten as:

$$\min_{\mathbf{W}} \left\| \mathbf{X} - \mathbf{X} \mathbf{W} \mathbf{W}^\top \right\|_F^2 \quad \text{s.t.} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_p. \quad (5)$$

In this formulation, the stored quantities are $\mathbf{W} \in \mathbb{R}^{r \times p}$ and $\mathbf{X} \mathbf{W} \in \mathbb{R}^{n \times p}$, with $\mathbf{X} \mathbf{W}$ corresponding to $\mathbf{C}$ in equation 4.

Theorem 1 (Optimal Solution to equation 5). *Let $\mathbf{X} \in \mathbb{R}^{n \times r}$ be zero-centered and $p < \min\{r, n\}$. The optimal basis $\mathbf{W}^*$ that minimizes equation 5 is given by the top- p eigenvectors of the sample covariance matrix $\mathbf{X}^\top \mathbf{X}$. Equivalently, $\mathbf{W}^*$ consists of the top- p right singular vectors of $\mathbf{X}$.*

Appendix A.1 provides the proof of Theorem 1. We now describe the two operational phases of RMM.

5.1 MERGING PHASE

The merging phase constructs a compact representation of all task-specific models through a shared basis. For each layer and each task-vector position (a fixed row in A_i^l or column in B_i^l), we gather the vectors from all n models into $\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$. We first compute the mean vector $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and zero-center $\mathbf{X}$ (i.e., $\mathbf{X} \leftarrow [\mathbf{x}_1 - \boldsymbol{\mu} \dots \mathbf{x}_n - \boldsymbol{\mu}]^\top$). We then perform SVD on the centered matrix $\mathbf{X} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top$ and select the top- p right singular vectors, yielding the optimal orthonormal basis $\mathbf{W}^*$ that minimizes the reconstruction error in equation 5, namely $\mathbf{W}^* = \mathbf{V}_{[:,1:p]} \in \mathbb{R}^{r \times p}$. The associated coefficient matrix $\mathbf{C}^* = \mathbf{X} \mathbf{W}^* \in \mathbb{R}^{n \times p}$ stores the linear-combination weights required for reconstruction; in particular, the i -th row of $\mathbf{C}^*$, $[c_{i1}, \dots, c_{ip}]$, contains the coefficients needed to reconstruct $\mathbf{x}_i$.

After this phase, the original task vectors $\{\mathbf{x}_i\}_{i=1}^n$ are discarded, and only the basis $\mathbf{W}^*$, the coefficients $\mathbf{C}^*$, and the mean vector $\boldsymbol{\mu}$ are retained for each task-vector position. Overall, the representation compresses the storage requirement from rn parameters, which would be needed to retain all task vectors, to $p(r+n) + r$ parameters for each task-vector position. The merging phase is performed offline before inference.

5.2 INFERENCE PHASE

Following common practice in model merging, we assume that the target task associated with each input is either specified explicitly by the user or determined automatically by an oracle router that assigns the input to the appropriate model index. Once the target task index i is known, the objective is to reconstruct its task vector $\mathbf{x}_i$, denoted $\hat{\mathbf{x}}_i$, from the compact representation $(\mathbf{W}^*, \mathbf{C}^*, \boldsymbol{\mu})$ stored at each task-vector position during the merging phase. The matrices $\mathbf{A}_i^l$ and $\mathbf{B}_i^l$ are then recovered by concatenating all reconstructed task vectors ($\hat{\mathbf{x}}_i$ vectors) across the relevant positions, namely the corresponding rows of $\mathbf{A}_i^l$ or columns of $\mathbf{B}_i^l$.

For each task-vector position, $\mathbf{W}^* \in \mathbb{R}^{r \times p}$ encodes the orthonormal basis directions, $\mathbf{C}^* \in \mathbb{R}^{n \times p}$ stores task-specific coefficients, and $\boldsymbol{\mu} \in \mathbb{R}^r$ denotes the mean vector over all task vectors. Reconstruction proceeds by linearly combining the basis directions with task-specific coefficients and then shifting the result by $\boldsymbol{\mu}$:

$$\hat{\mathbf{x}}_i = \sum_{j=1}^p c_{ij}^* \mathbf{w}_j^* + \boldsymbol{\mu} = \mathbf{C}_{[i,:]}^* \mathbf{W}^{*\top} + \boldsymbol{\mu}.$$

The resulting $\hat{\mathbf{x}}_i \in \mathbb{R}^r$ approximates the original task vector $\mathbf{x}_i$ that was discarded during merging. Because this reconstruction is performed independently for every task-vector position, including each row of $\mathbf{A}_i^l$ and each column of $\mathbf{B}_i^l$, and for all layers, it reconstructs the full low-rank delta for task-specific model i , denoted $\hat{\Delta}_i$. Algorithm 1 summarizes the complete merging and inference procedure. We next empirically evaluate the method against standard baselines.

Algorithm 1 Reversible Model Merging (RMM)

```

1: Merging Phase:
2: Require: Low-rank deltas  $\mathbf{A}_i^l$  and  $\mathbf{B}_i^l$ , compression rank  $r$ , number of basis vectors  $p$ 
3: for  $l = 1$  to  $L$  do
4:   for each task vector position (each row in  $\mathbf{A}_i^l$  or each column in  $\mathbf{B}_i^l$ ) do
5:     Gather task vectors  $\mathbf{x}_i$  of each model  $i$  ( $1 \leq i \leq n$ )
6:      $\mathbf{X} \leftarrow [\mathbf{x}_1 \dots \mathbf{x}_n]^\top \in \mathbb{R}^{n \times r}$ 
7:      $\boldsymbol{\mu} \leftarrow \sum_{i=1}^n \mathbf{x}_i \in \mathbb{R}^r$ 
8:      $\mathbf{X} \leftarrow [\mathbf{x}_1 - \boldsymbol{\mu} \dots \mathbf{x}_n - \boldsymbol{\mu}]^\top$ 
9:      $\mathbf{U}, \boldsymbol{\Sigma}, \mathbf{V}^\top \leftarrow \text{SVD}(\mathbf{X})$ 
10:     $\mathbf{W}^* \leftarrow \mathbf{V}_{[:,1:p]} \in \mathbb{R}^{r \times p}$ 
11:     $\mathbf{C}^* \leftarrow \mathbf{X} \mathbf{W}^* \in \mathbb{R}^{n \times p}$ 
12:    Store  $(\mathbf{W}^*, \mathbf{C}^*, \boldsymbol{\mu})$  and discard task vectors  $\{\mathbf{x}_1 \dots \mathbf{x}_n\}$ 
13:   end for
14: end for
15: Inference Phase:
16: Require: target task  $i$ , stored representation  $(\mathbf{W}^*, \mathbf{C}^*, \boldsymbol{\mu})$  for each task vector position
17: for  $l = 1$  to  $L$  do
18:   for each task vector position do
19:      $\hat{\mathbf{x}}_i = \mathbf{C}_{[i,:]}^* \mathbf{W}^{*\top} + \boldsymbol{\mu}$ 
20:   end for
21:    $\hat{\mathbf{A}}_i^l = \text{Concatenate}(\hat{\mathbf{x}}_i \text{ vectors across all row positions})$ 
22:    $\hat{\mathbf{B}}_i^l = \text{Concatenate}(\hat{\mathbf{x}}_i \text{ vectors across all column positions})$ 
23: end for
24:  $\hat{\Delta}_i = \{\hat{\mathbf{A}}_i^l \hat{\mathbf{B}}_i^l\}_{l=1}^L$ 
25: Inference with the reconstructed model  $\theta_{pre} + \hat{\Delta}_i$ 

```

6 EXPERIMENTS

To assess the effectiveness of RMM, we conduct experiments across multiple compression settings, tasks, and model architectures. The evaluation asks whether RMM can preserve the performance of low-rank task-specific models while reducing aggregate storage cost, and how this behavior compares with conventional merging baselines.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS-B	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	77.98	58.05	90.59	83.80
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.19	31.66
	TIES	13%	50.56	34.78	49.08	31.82	36.82	47.29	0.00	3.75	31.76
	DARE	13%	50.80	33.51	49.08	31.82	36.82	47.29	0.00	5.07	31.80
	RMM ($p = 2$)	50%	51.20	78.20	88.30	66.74	86.02	47.29	6.56	33.59	57.24
	RMM ($p = 3$)	69%	88.83	86.26	92.89	79.58	87.18	48.74	33.85	60.46	72.22
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	78.34	59.33	90.68	84.46
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.21	31.66
	TIES	13%	50.56	35.19	49.08	31.82	36.82	47.29	0.00	3.87	31.83
	DARE	13%	50.72	33.51	49.08	31.82	36.82	47.29	0.00	3.48	31.59
	RMM ($p = 2$)	44%	50.54	74.32	55.05	62.68	87.07	47.29	0.00	23.18	50.02
	RMM ($p = 3$)	59%	84.66	86.32	92.89	79.70	88.05	47.29	23.57	44.54	68.38
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	78.70	61.11	90.72	85.10
	TA	13%	50.61	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.13	49.08	31.82	36.82	47.29	0.00	3.80	31.82
	DARE	13%	50.58	33.51	49.08	31.82	36.82	47.29	0.00	5.22	31.79
	RMM ($p = 2$)	41%	50.53	73.10	49.08	58.48	86.98	47.29	0.00	17.50	47.87
	RMM ($p = 3$)	55%	74.14	86.20	92.43	78.83	89.00	47.29	13.16	33.70	64.34
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	79.78	60.16	90.74	85.30
	TA	13%	50.59	33.51	49.08	31.82	36.82	47.29	0.00	4.23	31.67
	TIES	13%	50.58	35.54	49.08	31.82	36.82	47.29	0.00	3.93	31.88
	DARE	13%	50.61	34.78	49.08	31.82	36.82	47.29	0.00	2.41	31.60
	RMM ($p = 2$)	39%	50.54	70.78	49.08	54.69	84.68	47.29	0.00	14.53	46.45
	RMM ($p = 3$)	52%	59.20	85.80	92.32	77.50	89.02	47.29	6.56	26.94	60.58

Table 1: GLUE benchmark performance when merging eight RoBERTa-base models compressed with PT-SVD across ranks $r = 16, 32, 64, 128$.

6.1 EXPERIMENTAL SETUP

We first evaluate RMM using eight RoBERTa-base models fine-tuned on eight distinct tasks (Zhuang et al., 2021). Each model is compressed individually with either PT-SVD or LoRA at target ranks $r \in \{16, 32, 64, 128\}$. After compression, RMM is applied to the resulting low-rank models and compared against standard merging baselines: Task Arithmetic (TA) (Ilharco et al., 2023), TIES-merging (Yadav et al., 2023), and DARE (Yu et al., 2024). We also report the performance of the individual compressed models before merging as an upper reference point. Appendix A.2 provides additional experiments, including results on OPT-1.3b (Zhang et al., 2022) and on ViT-B/32 (Radford et al., 2021) for vision tasks, with varying numbers of tasks.

Following the GLUE benchmark protocol (Wang et al., 2018), each task is evaluated with its standard metric. QNLI (question–answering inference), MRPC (paraphrase detection), SST-2 (sentiment classification), MNLI (multi-genre natural language inference), QQP (duplicate question detection), and RTE (textual entailment) are measured by accuracy; CoLA (linguistic acceptability) is measured with Matthews correlation, and STS-B (semantic textual similarity) with Pearson correlation. We report the average score across tasks as the overall performance measure. For each method, we also compute relative storage cost, defined as the fraction of parameters retained relative to storing all individual task-specific adapters. Baselines that store a single merged model have fixed cost $\frac{r}{rn} = \frac{1}{n}$. For RMM with p basis components, the relative storage cost is $\frac{p(r+n)+r}{rn}$, as derived in Section 5.1.

6.2 RESULTS AND DISCUSSION

Tables 1 and 2 report RMM performance relative to the baselines under PT-SVD and LoRA compression, respectively. Across both compression regimes and all target ranks $r \in \{16, 32, 64, 128\}$, RMM consistently exceeds TA, TIES, and DARE by a wide margin. At low ranks ($r = 16, 32$), the baselines fail catastrophically, reaching only 31–42% average score and performing especially poorly on structurally sensitive tasks such as CoLA and STS-B. By contrast, RMM with $p = 2$ yields substantial improvements, including 57.24% under PT-SVD and 51.41% under LoRA at $r = 16$, while RMM with $p = 3$ strengthens the recovery further, reaching 72.22% and 58.64% at $r = 16$ and thereby substantially narrowing the gap to the individual compressed models. At higher ranks ($r = 64, 128$), where compression error is smaller, RMM continues to dominate the baselines: with $p = 3$, it obtains 64.34% and 60.58% under PT-SVD, and 55.40% and 61.35% under LoRA,

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	RTE	COLA	STS	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	68.23	53.92	88.71	77.71
	TA	13%	50.59	33.51	72.13	34.79	64.46	47.29	0.00	21.81	40.57
	TIES	13%	50.63	33.68	74.31	35.64	67.45	47.29	0.00	26.38	41.92
	DARE	13%	50.63	33.51	73.74	35.97	65.35	47.29	0.00	21.62	41.01
	RMM ($p = 2$)	50%	51.13	34.14	90.71	60.48	84.41	47.29	0.00	43.14	51.41
	RMM ($p = 3$)	69%	55.92	38.14	93.12	82.95	87.30	47.29	6.56	57.81	58.64
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	62.82	53.12	88.53	76.54
	TA	13%	50.91	68.06	54.70	33.77	65.50	55.96	0.00	-0.48	41.05
	TIES	13%	51.03	68.06	56.77	34.24	66.27	58.12	0.00	1.02	41.94
	DARE	13%	50.96	64.64	63.07	35.12	63.66	55.60	0.00	1.01	41.76
	RMM ($p = 2$)	44%	52.08	65.39	81.54	62.46	77.06	57.40	0.00	9.95	50.73
	RMM ($p = 3$)	59%	54.62	69.97	92.20	81.29	85.97	57.40	4.64	24.08	58.77
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	67.15	58.31	88.70	78.25
	TA	13%	54.75	68.52	77.29	37.24	36.82	47.29	0.00	-3.53	39.80
	TIES	13%	54.82	68.75	80.73	37.67	36.82	47.29	0.00	-2.48	40.45
	DARE	13%	53.71	68.58	78.56	38.33	36.82	47.29	0.00	-4.28	39.88
	RMM ($p = 2$)	41%	55.74	69.28	87.27	42.81	77.72	47.29	0.00	4.91	48.13
	RMM ($p = 3$)	55%	59.07	70.14	90.83	79.11	82.06	47.29	0.00	14.70	55.40
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	57.04	59.35	89.68	77.71
	TA	13%	51.49	69.22	54.47	37.16	63.27	47.29	0.00	-1.74	40.15
	TIES	13%	51.91	68.93	57.34	37.81	63.36	47.29	0.00	-0.25	40.80
	DARE	13%	51.97	68.99	58.60	35.64	63.33	47.29	0.00	-0.44	40.67
	RMM ($p = 2$)	39%	57.18	67.83	88.53	49.54	84.64	47.29	0.00	28.00	52.88
	RMM ($p = 3$)	52%	59.13	39.19	92.09	80.08	86.39	47.29	15.45	71.18	61.35

Table 2: GLUE benchmark performance when merging eight RoBERTa-base models compressed with LoRA across ranks $r = 16, 32, 64, 128$.

whereas single-model baselines remain near 40%. Even the $p = 2$ variant retains clear advantages, indicating that the reversible formulation mitigates the destructive interference that undermines prior methods.

We also observe that the relative advantage of RMM over the baselines diminishes as rank increases. A plausible explanation is that higher-rank task representations occupy higher-dimensional spaces, becoming more expressive but also less mutually aligned, which makes their compression into a shared low-dimensional basis more difficult. Overall, the results show that RMM delivers a consistent advantage across both compression regimes and all ranks: it preserves stronger task performance while incurring additional storage, thereby providing a tunable accuracy-storage trade-off through the number of reversible components. This storage overhead pertains to **offline memory**, which is typically CPU-resident and inexpensive, whereas task performance directly determines runtime quality. Under this operating regime, a modest increase in CPU-side storage is often justified by substantially improved accuracy.

6.3 SCALABILITY OF RMM

We further examine the scalability of RMM as the number of models, n , increases. Figures 4 and 5 plot the storage ratio, measured relative to storing all individual task-specific models, as a function of the number of tasks for RMM with $p = 2$ and $p = 3$, respectively. The results indicate favorable scaling: as the task collection grows, the relative storage cost decreases markedly. This pattern reflects the central advantage of reversible merging, namely that storage grows sublinearly with the number of task-specific models. The sublinearity follows directly from the RMM parameterization: once the basis has been fixed, incorporating a new task requires only an additional row in the coefficient matrix. Conventional alternatives, by contrast, either store a single merged model, thereby sacrificing performance, or retain each task-specific model independently, thereby scaling linearly. These findings underscore the practical value of RMM in large-scale multi-task, continual, or federated settings, where hundreds of models may need to be managed efficiently without sacrificing task performance.

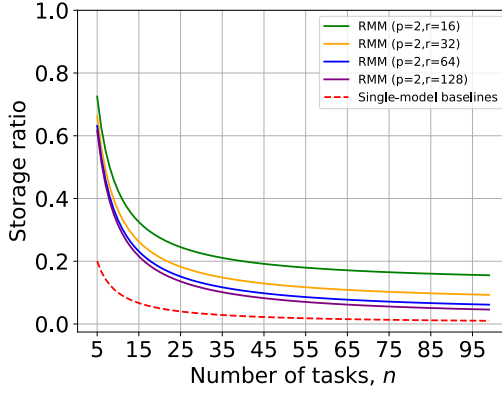


Figure 4: Scalability of RMM with $p = 2$. The storage ratio denotes the percentage of memory required relative to storing all models independently.

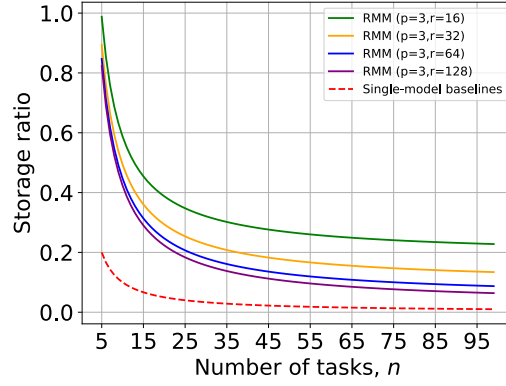


Figure 5: Scalability of RMM with $p = 3$. The storage ratio denotes the percentage of memory required relative to storing all models independently.

7 CONCLUSION

This work introduced a principled framework for reversible model merging that redefines how multiple fine-tuned models can be stored, merged, and reconstructed. Rather than forcing all task-specific knowledge into a single merged model, the framework relaxes this constraint by retaining a compact set of intermediate representations that is substantially smaller than the original model collection while remaining more expressive than a single aggregate. This design enables accurate reconstruction of task-specific models while substantially reducing storage overhead. The method provides a controllable trade-off between performance and storage efficiency, particularly for low-rank compressed models. By representing fine-tuned models through compact bases and reversible mappings, we show that models can be merged, reconstructed, and deployed with high accuracy under strict memory budgets, while allowing practitioners to adapt compression strength and reconstruction fidelity to application-specific requirements.

Overall, the framework moves beyond heuristic or one-shot merging procedures by formalizing model merging as a structured and reversible process. This perspective broadens the design space for efficient multi-task deployment and opens promising directions for future work on performance-aware compression and reconstruction strategies.

REFERENCES

- Mircea Cimpoi, Subhansu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3606–3613, 2014.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300/>.
- Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodola. Task singular vectors: Reducing task interference in model merging. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 18695–18705, 2025.

- Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.
- Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2 edition, 2012.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emr-merging: Tuning-free high-performance model merging. *Advances in Neural Information Processing Systems*, 37:122741–122769, 2024.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=6t0Kwf8-jrj>.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=FCnohuR6AnM>.
- Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *2013 IEEE International Conference on Computer Vision Workshops*, pp. 554–561, 2013. doi: 10.1109/ICCVW.2013.77.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011. URL http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf.
- Bowen Ping, Shuo Wang, Hanqing Wang, Xu Han, Yuzhuang Xu, Yukun Yan, Yun Chen, Baobao Chang, Zhiyuan Liu, and Maosong Sun. Delta-come: Training-free delta-compression with mixed-precision for large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 31056–31077. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/37664246a1e07e212ddacea6e5a523f2-Paper-Conference.pdf.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 8748–8763. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/radford21a.html>.
- Simo Ryu, Seunghyun Seo, and Jaejun Yoo. Efficient storage of fine-tuned models via low-rank approximation of weight residuals. *arXiv preprint arXiv:2305.18425*, 2023.
- Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *The 2011 International Joint Conference on Neural Networks*, pp. 1453–1460, 2011. doi: 10.1109/IJCNN.2011.6033395.

- Dennis Tang, Prateek Yadav, Yi-Lin Sung, Jaehong Yoon, and Mohit Bansal. LoRA merging with SVD: Understanding interference and preserving performance. In *ICML 2025 Workshop on Reliable and Responsible Foundation Models*, 2025. URL <https://openreview.net/forum?id=t9FrMviTaP>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (eds.), *Proceedings of the 2018 EMNLP Workshop Black-boxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446. URL <https://aclanthology.org/W18-5446/>.
- Xin Wang, Samiul Alam, Zhongwei Wan, Hui Shen, and Mi Zhang. SVD-LLM v2: Optimizing singular value truncation for large language model compression. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4287–4296, Albuquerque, New Mexico, April 2025a. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.217. URL <https://aclanthology.org/2025.naacl-long.217/>.
- Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *The Thirteenth International Conference on Learning Representations*, 2025b. URL <https://openreview.net/forum?id=LNyIUouhdt>.
- Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EqoKRSR5Pa>.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pp. 23965–23998. PMLR, 2022.
- Jianxiong Xiao, James Hays, Krista A. Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 3485–3492, 2010. doi: 10.1109/CVPR.2010.5539970.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=xtaX3WYCj1>.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=nZP6NgD3QY>.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- Juzheng Zhang, Jiacheng You, Ashwinee Panda, and Tom Goldstein. LoRI: Reducing cross-task interference in multi-task low-rank adaptation. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=b8cW86QcOD>.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

Ziyu Zhao, Tao Shen, Didi Zhu, Zexi Li, Jing Su, Xuwu Wang, and Fei Wu. Merging loRAs like playing LEGO: Pushing the modularity of loRA to extremes through rank-wise clustering. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=j6f5sbpAllN>.

Shenghe Zheng, Hongzhi Wang, Chenyu Huang, Xiaohui Wang, Tao Chen, Jiayuan Fan, Shuyue Hu, and Peng Ye. Decouple and orthogonalize: A data-free framework for lora merging. *arXiv preprint arXiv:2505.15875*, 2025.

Liu Zhuang, Lin Wayne, Shi Ya, and Zhao Jun. A robustly optimized BERT pre-training approach with post-training. In Sheng Li, Maosong Sun, Yang Liu, Hua Wu, Kang Liu, Wanxiang Che, Shizhu He, and Gaoqi Rao (eds.), *Proceedings of the 20th Chinese National Conference on Computational Linguistics*, pp. 1218–1227, Huhhot, China, August 2021. Chinese Information Processing Society of China. URL <https://aclanthology.org/2021.ccl-1.108/>.

A APPENDIX

A.1 PROOF OF THEOREM 1

Let $\mathbf{A} = \mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top$. Invoking the Frobenius–trace identity, $\|\mathbf{A}\|_F^2 = \text{Tr}(\mathbf{A}^\top \mathbf{A})$, where $\text{Tr}(\cdot)$ denotes the trace operator on square matrices, the reconstruction objective can be expanded as:

$$\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 = \text{Tr}((\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)).$$

The matrix product appearing inside the trace admits the following algebraic expansion:

$$\begin{aligned} (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) &= (\mathbf{X}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top)(\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top) \\ &= \mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top - \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X} \\ &\quad + \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}\mathbf{W}^\top. \end{aligned}$$

Taking the trace of this expanded expression, and subsequently applying both trace cyclicity and the orthonormality constraint $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p$, gives:

$$\begin{aligned} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{W}\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - 2\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) + \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}) \\ &= \text{Tr}(\mathbf{X}^\top \mathbf{X}) - \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}). \end{aligned}$$

Because $\text{Tr}(\mathbf{X}^\top \mathbf{X})$ is invariant with respect to $\mathbf{W}$, minimization of $\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2$ is equivalent to maximization of $\text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W})$. Consequently,

$$\min_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2 \iff \max_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_p} \text{Tr}(\mathbf{W}^\top \mathbf{X}^\top \mathbf{X}\mathbf{W}).$$

By the Rayleigh quotient theorem (Horn & Johnson, 2012), the maximizing subspace is obtained by selecting the columns of $\mathbf{W}$ to be the top- p eigenvectors of $\mathbf{X}^\top \mathbf{X}$. This choice coincides with the principal component analysis (PCA) solution, which identifies the p -dimensional subspace that maximizes projected variance and, equivalently, minimizes squared reconstruction error. Furthermore, if $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ denotes the SVD of $\mathbf{X}$, then $\mathbf{X}^\top \mathbf{X} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$, so the eigenvectors of $\mathbf{X}^\top \mathbf{X}$ are precisely the right singular vectors, i.e., the columns of $\mathbf{V}$. Hence, the PCA solution is equivalently obtained by retaining the top- p right singular vectors of $\mathbf{X}$; in practice, this SVD-based computation is numerically more stable, and frequently more computationally attractive, than explicitly constructing $\mathbf{X}^\top \mathbf{X}$, particularly when r is large.

A.2 ADDITIONAL RESULTS

This section provides extended empirical results for two additional pre-trained architectures, OPT-1.3b and ViT-B/32, thereby examining whether the observed behavior of RMM persists beyond the primary RoBERTa-base evaluation. For **OPT-1.3b**, the evaluation comprises six natural-language-understanding tasks drawn from GLUE and related benchmarks: QNLI, MRPC, SST-2, MNLI,

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	BOOLQ	Average
16	No merging	100%	91.74	71.54	95.30	84.19	88.14	71.47	83.73
	Task-Arithmetic	17%	50.34	37.16	58.49	34.43	63.30	41.87	47.60
	TIES	17%	49.79	36.75	61.24	35.57	63.55	40.67	47.93
	DARE	17%	49.95	37.68	56.08	34.51	63.39	42.78	47.40
	RMM ($p = 2$)	63%	88.12	44.00	93.58	73.06	85.63	59.69	74.01
	RMM ($p = 3$)	85%	91.58	49.04	95.64	80.31	87.52	65.50	78.27
32	No merging	100%	92.06	72.17	95.41	84.60	89.17	72.45	84.31
	Task-Arithmetic	17%	50.28	37.22	58.94	34.57	63.33	41.93	47.71
	TIES	17%	49.77	36.93	61.93	35.69	63.65	40.73	48.12
	DARE	17%	50.39	40.87	56.19	34.96	63.59	43.33	48.22
	RMM ($p = 2$)	56%	86.66	41.91	91.74	71.75	85.68	56.39	72.36
	RMM ($p = 3$)	76%	91.85	44.81	94.84	80.21	88.43	63.52	77.28
64	No merging	100%	92.26	72.12	95.41	84.64	90.07	72.84	84.56
	Task-Arithmetic	17%	50.28	37.28	59.06	34.56	63.34	42.08	47.77
	TIES	17%	49.75	37.04	61.81	35.84	63.74	40.76	48.16
	DARE	17%	50.16	40.52	58.83	35.59	63.35	42.54	48.50
	RMM ($p = 2$)	53%	83.87	42.72	89.45	69.76	85.70	53.43	70.82
	RMM ($p = 3$)	71%	92.07	41.97	94.50	79.70	88.73	61.96	76.49
128	No merging	100%	92.35	72.41	95.41	84.67	90.93	73.09	84.81
	Task-Arithmetic	17%	50.32	37.39	58.94	34.63	63.37	42.08	47.79
	TIES	17%	49.79	37.04	62.04	35.99	63.74	40.55	48.19
	DARE	17%	50.03	38.14	57.91	34.72	63.31	42.11	47.70
	RMM ($p = 2$)	52%	81.38	42.78	84.75	67.66	83.22	51.99	68.63
	RMM ($p = 3$)	69%	92.28	41.80	93.69	79.42	88.76	59.69	75.94

Table 3: GLUE benchmark performance when merging six OPT-1.3b models compressed with PT-SVD across ranks $r = 16, 32, 64, 128$.

r	Method	Storage	MNIST	GTSRB	SVHN	Cars	EuroSAT	DTD	SUN397	Average
16	No merging	100%	99.61	97.22	97.06	69.20	99.67	75.21	72.72	87.24
	Task-Arithmetic	14%	47.19	28.29	38.64	56.27	57.52	41.38	62.92	47.46
	TIES	14%	60.14	34.52	49.03	58.55	61.70	43.67	62.87	52.93
	DARE	14%	53.80	31.00	44.96	57.60	56.93	43.99	62.42	50.10
	RMM ($p = 2$)	55%	99.16	88.52	94.89	66.61	97.44	60.64	67.86	82.16
	RMM ($p = 3$)	76%	99.58	95.65	96.54	68.30	98.93	67.18	69.50	85.10
32	No merging	100%	99.65	98.45	97.35	73.04	99.78	78.72	75.52	88.93
	Task-Arithmetic	14%	47.90	28.31	38.68	56.24	57.52	41.38	63.00	47.58
	TIES	14%	61.25	35.28	49.95	58.80	62.41	44.04	63.05	53.54
	DARE	14%	53.05	31.04	46.34	58.54	57.74	42.87	62.87	50.35
	RMM ($p = 2$)	49%	98.98	88.23	94.29	68.32	96.81	61.91	68.36	82.41
	RMM ($p = 3$)	67%	99.50	96.53	96.68	70.90	98.93	70.43	71.23	86.31
64	No merging	100%	99.67	98.68	97.41	75.76	99.85	79.15	77.73	89.75
	Task-Arithmetic	14%	47.99	28.30	38.71	56.21	57.59	41.38	63.22	47.63
	TIES	14%	61.83	35.93	50.28	59.08	63.07	44.15	63.17	53.93
	DARE	14%	54.78	32.79	45.05	58.23	61.30	42.87	62.77	51.11
	RMM ($p = 2$)	46%	98.46	84.43	92.83	69.36	95.59	62.39	69.77	81.83
	RMM ($p = 3$)	62%	99.45	96.14	96.33	72.84	98.56	72.07	73.85	87.03
128	No merging	100%	99.68	98.75	97.46	76.99	99.85	79.04	78.59	90.05
	Task-Arithmetic	14%	48.00	28.31	38.73	56.21	57.63	41.44	63.35	47.67
	TIES	14%	62.08	36.36	50.48	59.23	63.41	44.68	63.25	54.21
	DARE	14%	55.16	32.07	45.47	58.55	61.85	43.09	62.85	51.29
	RMM ($p = 2$)	44%	97.22	78.56	90.63	70.22	94.00	61.49	71.03	80.45
	RMM ($p = 3$)	59%	99.28	94.59	95.95	74.24	98.19	71.86	76.02	87.16

Table 4: Vision-task performance when merging seven ViT-B/32 models compressed with PT-SVD across ranks $r = 16, 32, 64, 128$.

QQP, and BoolQ (Clark et al., 2019). For **ViT-B/32**, the evaluation shifts to vision classification across seven heterogeneous datasets: MNIST (LeCun et al., 1998), GTSRB (Stallkamp et al., 2011), SVHN (Netzer et al., 2011), Cars (Krause et al., 2013), EuroSAT (Helber et al., 2019), DTD (Cimpoi et al., 2014), and SUN397 (Xiao et al., 2010).

Table 3 reports the outcomes for six OPT-1.3b models compressed with PT-SVD. Across ranks, both RMM variants ($p = 2$ and $p = 3$) consistently exceed the baseline methods, and the $p = 3$ configuration narrows the discrepancy relative to the original individual models while still requiring substantially fewer parameters. Table 4 presents the corresponding results for seven ViT-B/32

models. In the vision setting, the advantage of RMM is even more pronounced: with $p = 3$, the method achieves an average accuracy less than 3% below the original-model average while reducing storage to only 59% of the cost of retaining all models at $r = 128$. These results further substantiate the claim that RMM can reconstruct the original compressed models with high fidelity while simultaneously delivering substantial storage efficiency.

A.3 RESULTS FOR 5 TASKS

For the experiments summarized in Section 4, RoBERTa-base serves as the pretrained backbone, and the merging baselines are evaluated on five GLUE tasks: QNLI, MRPC, SST-2, MNLI, and QQP. Figures 1 and 2 in Section 4 provide the aggregate visualization of these results. For completeness, Table 5 supplies the tabular counterpart to Figure 1, reporting the LoRA fine-tuning results together with the improvements obtained by RMM, whereas Table 6 supplies the counterpart to Figure 2 for PT-SVD compressed models.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	61.38	83.48	94.50	83.90	87.56	82.16
	Task-Arithmetic	20%	50.61	33.51	73.85	34.23	64.58	51.36
	TIES	20%	50.65	33.51	75.57	35.83	66.97	52.51
	DARE	20%	50.61	33.51	79.36	34.70	63.40	52.32
	RMM ($p = 2$)	73%	59.64	68.64	93.00	82.68	86.89	78.17
32	No merging	100%	60.13	81.28	93.81	84.46	88.18	81.57
	Task-Arithmetic	20%	51.18	67.30	58.83	34.33	68.56	56.04
	TIES	20%	51.36	68.17	59.52	34.76	71.46	57.05
	DARE	20%	50.83	64.87	68.12	35.54	68.93	57.66
	RMM ($p = 2$)	66%	57.72	67.54	92.55	81.80	86.13	77.15
64	No merging	100%	59.62	84.12	94.15	85.20	88.75	82.37
	Task-Arithmetic	20%	54.74	68.93	80.28	35.70	36.82	55.29
	TIES	20%	55.26	68.93	78.78	36.76	36.82	55.31
	DARE	20%	54.71	68.81	74.54	38.72	36.82	54.72
	RMM ($p = 2$)	63%	59.82	67.48	91.51	83.21	84.41	77.29
128	No merging	100%	61.58	85.04	93.58	85.71	89.68	83.12
	Task-Arithmetic	20%	52.52	68.70	54.82	37.83	63.46	55.47
	TIES	20%	52.55	69.04	57.80	38.91	63.72	56.40
	DARE	20%	55.04	69.62	54.47	38.92	64.12	56.43
	RMM ($p = 2$)	62%	60.21	34.96	91.86	78.59	86.65	70.45

Table 5: GLUE benchmark performance when merging five RoBERTa-base models compressed with LoRA across ranks $r = 16, 32, 64, 128$.

r	Method	Storage	QNLI	MRPC	SST-2	MNLI	QQP	Average
16	No merging	100%	92.02	85.68	93.69	85.21	87.19	88.76
	Task-Arithmetic	20%	53.03	35.19	49.08	31.82	36.82	41.19
	TIES	20%	55.46	54.32	49.08	31.82	37.21	45.58
	DARE	20%	53.69	58.67	49.08	31.82	37.08	46.07
	RMM ($p = 2$)	73%	79.72	85.16	90.83	76.58	86.09	83.68
32	No merging	100%	92.31	85.86	94.15	86.38	88.62	89.46
	Task-Arithmetic	20%	53.23	35.59	49.08	31.82	36.82	41.31
	TIES	20%	55.78	55.19	49.08	31.82	37.62	45.90
	DARE	20%	55.89	33.74	49.20	31.82	36.85	41.50
	RMM ($p = 2$)	66%	63.87	82.78	91.28	77.12	87.05	80.42
64	No merging	100%	92.51	86.03	94.38	87.18	90.15	90.05
	Task-Arithmetic	20%	53.49	35.88	49.08	31.82	36.82	41.42
	TIES	20%	56.14	55.94	49.08	31.82	38.08	46.21
	DARE	20%	53.51	33.68	49.08	31.82	36.82	40.98
	RMM ($p = 2$)	63%	56.12	81.22	91.06	76.74	88.07	78.64
128	No merging	100%	92.68	86.14	94.50	87.33	91.05	90.34
	Task-Arithmetic	20%	53.60	36.06	49.08	31.82	36.82	41.48
	TIES	20%	56.40	56.35	49.08	31.82	38.61	46.45
	DARE	20%	50.52	34.96	49.08	31.82	36.92	40.66
	RMM ($p = 2$)	62%	54.11	79.19	90.60	74.49	87.46	77.17

Table 6: GLUE benchmark performance when merging five RoBERTa-base models compressed with PT-SVD across ranks $r = 16, 32, 64, 128$.